
Generators

Normal Function

```
def numbers():
```

```
    return 10
```

```
    return 20
```

```
    return 30
```

```
print(numbers()) #10
```

```
print(numbers()) #10
```

```
print(numbers()) #10
```

- A normal function uses the return keyword to return a value to the caller.
- Once the return statement is executed, the function stops immediately.
- The remaining return statements are not executed.
- In some situations, we need to generate multiple values one by one.
- A normal function stops after executing the first return statement.
- To solve this problem, **Python provides generators.**

Generator

It is a type of a iterator object that produces values one at a time when required instead of storing all values in memory at once.

Generators are useful when working with large sequences of data because they help in saving memory.

Generators can be created using:

1. **Generator Functions**
 2. **Generator Expressions**
-

Generators using Generator Function

- A generator function is a special type of function that uses the **yield keyword** to generate values one at a time.
- When a generator function is called, it returns a **generator object**.
- The function pauses after each yield statement and resumes execution when the next value is requested using a `__next__()` method or `next()` function.

Example:

```
def numbers():
```

```
    yield 10
```

```
    yield 20
```

```
    yield 30
```

```
g = numbers()
```

```
print(g.__next__()) #10
```

```
print(g.__next__()) #20
```

```
print(g.__next__()) #30
```

```
print(g.__next__()) #StopIteration
```

- Here `g` is a generator object and `numbers()` is a generator function.
 - `yield 10` returns the first value.
 - Then the function pauses its execution.
 - When `next()` function or `__next__()` method is called again, execution resumes from the next yield statement.
 - Values are generated one by one.
 - After all values are generated, calling `__next__()` again will raise a `StopIteration` exception.
-

Generators using Generator Expression

A generator expression is a short and simple way to create generators, similar to list comprehension.

It uses parentheses () instead of square brackets [].

Syntax : `generator_object = (expression for item in iterable)`

Example

```
g = ( i for i in range(1,6) )
```

```
print(g.__next__()) #1
```

```
print(g.__next__()) #2
```

```
print(g.__next__()) #3
```

```
print(g.__next__()) #4
```

```
print(g.__next__()) #5
```

```
print(g.__next__()) #StopIteration
```

- Here g is a generator object and (i for i in range(1,6)) is generator expression.
 - The generator does not store all values at once.
 - Each time `__next__()` is called, the next value is generated.
 - After all values are generated, calling `__next__()` again will raise a StopIteration exception.
-

Example Programs:

1. Write a Python program to generate the first 5 natural numbers using a generator function.

```
def natural_numbers():  
    for i in range(1,6):  
        yield i
```

```
g = natural_numbers()
```

```
print(next(g)) #1  
print(next(g)) #2  
print(next(g)) #3  
print(next(g)) #4  
print(next(g)) #5
```

2. Write a Python program to generate square of a list numbers using a generator function.

```
def squares( nums ):  
    for i in nums:  
        yield i*i
```

```
list = [2, 6, 8, 9, 6]  
g = squares( list )
```

```
print(g.__next__()) #4  
print(g.__next__()) #36  
print(g.__next__()) #64  
print(g.__next__()) #81  
print(g.__next__()) #36
```

3. Write a Python program to generate even numbers from 1 to 10 using a generator function.

```
def even_numbers():  
    for i in range(1,11):  
        if i % 2 == 0:  
            yield i
```

```
g = even_numbers()
```

```
print(next(g)) #2  
print(next(g)) #4  
print(next(g)) #6  
print(next(g)) #8  
print(next(g)) #10
```

4. Write a Python program to generate the USN of students using a generator function.

```
def usn(institute, year, course):  
    num = 1  
    while True:  
        yield str(num) + institute + str(year) + course  
        num += 1
```

```
g = generate_usn("PYSP", 26, "PFS")
```

```
print(next(g)) # 1PYSP26PFS  
print(next(g)) # 2PYSP26PFS  
print(next(g)) # 3PYSP26PFS  
print(next(g)) # 3PYSP26PFS  
print(next(g)) # 4PYSP26PFS
```
